

1 Kitchen Intelligence

1.1 Current Project Status

Release candidate: Final validated build

Checkpoint: 276921cc

Project type: Authenticated full-stack web application

Status date: August 21, 2026

1.2 Executive summary

Kitchen Intelligence is a premium AI-powered meal planning and pantry management application. The current release candidate implements the core product loop from onboarding and pantry capture through personalized meal generation, weekly planning, shopping consolidation, cooking reconciliation, privacy controls, and protected administration.

The application is structured as a React and Tailwind frontend with an Express and tRPC backend, Manus authentication, a Drizzle/MySQL data layer, persistent user-scoped records, and explicit provider boundaries for AI, vision, retailer discovery, billing, storage, and monitoring.

1.3 Implemented product capabilities

Area	Current status
Landing and trust surface	Premium responsive landing page with workflow explanation, pricing disclosure, FAQ, Terms destination, privacy language, support contact, and AI disclosure.
Onboarding	Multi-step authenticated onboarding for household size, diet, allergies, cuisines, dislikes, cooking skill, cooking time, budget, and meal priorities. Preferences persist to the user profile.
Pantry	Persistent inventory with ingredient name, category, quantity, unit, expiration, confidence, source, location, inline editing, search, consumption, replenishment, deletion, and quantity adjustment.
Pantry scanning	Multi-image scan workflow with configurable confidence threshold, candidate review, correction, removal, deduplication, and explicit confirmation before inventory commit.
Meal intelligence	Structured AI meal generation and modification with pantry context, profile context, occasion, cooking constraints, budget, allergy hard filters, recommendation scoring, missing-ingredient

Area	Current status
	penalties, interaction history, and usage tracking.
Recipes	Detailed meal results with serving scaling, substitution guidance, pantry matches, missing ingredients, save/favorite, feedback, share, regenerate, modify, cook, and undo-cook controls.
Weekly planning	Day-by-day planning with consolidated shopping items and explicit leftover carry-forward decisions.
Shopping	User-scoped shopping list with missing-ingredient detection, quantity consolidation, serving-aware scaling, meal associations, manual add, inline quantity/unit update, check-off, and removal.
Cooking reconciliation	Editable consumption quantities, pantry decrement after cooking, persisted reversible snapshot, and user-scoped undo.
Privacy and account controls	Account deletion confirmation, scan-history deletion, recommendation-feedback deletion, AI disclosure, support path, and fail-safe error states.
Billing and entitlements	Server-side free-plan limits and entitlement guard for AI generation, pantry scanning, weekly planning, and meal modification. Checkout remains explicitly disabled until a verified billing provider is connected.
Observability	Protected admin view with users, meals, pantry items, AI operations, subscriptions, failed operations, failure rate, funnel conversion, analytics event volume, and subscription plan/status mix.
Provider boundaries	Structured AI and vision interfaces, usage metadata extraction, disabled-by-default retailer adapter, and documented billing, storage, monitoring, and integration boundaries.

1.4 Technical implementation

The project uses React 19, Tailwind CSS, Express, tRPC, Drizzle ORM, MySQL/TiDB-compatible schema definitions, Manus OAuth, and Vitest. The backend keeps user isolation in authenticated procedures and database helpers. AI operations are represented through structured schemas rather than free-form persistence, and provider responses record model, input tokens, output tokens, estimated cost, success state, and operation name.

Deterministic domain utilities cover canonical ingredient matching, safe structured parsing, recommendation scoring, entitlement decisions, recipe serving scaling, and shopping consolidation. These utilities provide a stable test seam independent of external AI providers.

1.5 Validation completed

Gate	Result
TypeScript check	Passed with pnpm check.
Automated tests	Passed: 4 test files and 17 tests.
Production build	Passed with pnpm build.
Responsive verification	Desktop and mobile previews captured during implementation.
New-user QA documentation	Added to docs/TESTING.md, covering onboarding through privacy controls.
Release checkpoint	Saved as manus-webdev://276921cc.

The build reports a non-blocking bundle-size warning for the main JavaScript chunk. The development server is running in the managed project environment.

1.6 Launch prerequisites and boundaries

The application is intentionally honest about external dependencies. Production launch requires verified credentials and configuration for the AI and vision providers, a billing provider for paid checkout and subscription lifecycle events, an optional retailer catalog provider for product discovery, and production error monitoring. Until those services are connected, the application uses explicit provider boundaries, fail-closed billing behavior, and does not invent product prices, availability, retailer links, testimonials, or customer reviews.

The project contains a detailed audit in docs/LAUNCH_AUDIT.md, along with architecture, AI system, billing, cost, testing, environment, analytics, and provider-integration references. These documents distinguish implemented behavior from provider-dependent launch work.

1.7 Recommended next steps

1. Connect and verify production AI, vision, billing, retailer, and monitoring providers, including rate limits and webhook lifecycle handling.
2. Run the documented fresh-account QA flow with real credentials and representative pantry images, then capture any provider-specific failure behavior.

3. Add production alerting, bundle code-splitting, and a monitored analytics dashboard before public rollout.